

```

/**
 * quick-fixes-patch.ts
 *
 * Patches for Fixes #4, #6, #7, #8
 * These are small targeted changes – apply each to the named file.
 *
 * =====
 * FIX #4 – content-grinder burns daily token budget in 3 hours
 * FILE: server/services/content-grinder.ts
 * =====
 *
 * Add a 30-second delay between each video processed, and cap the grinder
 * to processing max 20 videos per cycle. This spreads the 100k token budget
 * across the full day instead of exhausting it in 3 hours.
 *
 * BEFORE:
 *   for (const video of videos) {
 *     await processVideo(video);
 *   }
 *
 * AFTER:
 */

const GRINDER_MAX_PER_CYCLE = 20;
const GRINDER_INTER_VIDEO_DELAY_MS = 30_000; // 30 seconds

async function runGrinderCyclePatched(videos: any[]) {
  const batch = videos.slice(0, GRINDER_MAX_PER_CYCLE);

  for (const video of batch) {
    await processVideo(video);
    // Spread token usage across the day – don't burn budget in one cycle
    await new Promise(r => setTimeout(r, GRINDER_INTER_VIDEO_DELAY_MS));
  }
}

// Also separate YouTube metadata UPDATE calls from fresh UPLOAD quota.
// In the content-grinder, before calling youtube.videos.update():
//
// BEFORE:
//   await youtube.videos.update({ ... });
//
// AFTER:
//   const quotaRemaining = await storage.getQuotaRemaining(userId);
//   if (quotaRemaining < 200) {
//     log.warn('[ContentGrinder] Insufficient quota for metadata update – deferring');
//     await storage.deferMetadataUpdate(video.id);
//     continue;

```

```

//    }
//    await youtube.videos.update({ ... });

declare function processVideo(v: any): Promise<void>;

/**
 * =====
 * FIX #6 – indexAllChannelVideos called every 6 seconds (runaway loop)
 * FILE: wherever indexAllChannelVideos is called on a timer/interval
 * =====
 *
 * The vault already has an internal idempotency guard, but the CALLER
 * is in a loop that fires every 6 seconds. Fix the caller – add a
 * minimum 10-minute gap between calls at the call site.
 *
 * BEFORE (in whatever service is calling this):
 *   setInterval(() => {
 *     videoVault.indexAllChannelVideos(userId);
 *   }, 6000);
 *
 * AFTER:
 */

let lastIndexCallMs = 0;
const INDEX_MINIMUM_GAP_MS = 10 * 60 * 1000; // 10 minutes minimum between calls

function indexAllChannelVideosThrottled(videoVault: any, userId: string) {
  const now = Date.now();
  if (now - lastIndexCallMs < INDEX_MINIMUM_GAP_MS) {
    // Don't log – this is expected throttling, not an error
    return;
  }
  lastIndexCallMs = now;
  videoVault.indexAllChannelVideos(userId);
}

/**
 * =====
 * FIX #7 – TikTok phantom user burning AI slots in growth-flywheel
 * FILE: server/services/growth-flywheel.ts (and infinite-evolution-engine.ts)
 * =====
 *
 * userId "tiktok_-000hfXLzkfKJGE24wvR..." appears in multiple modules.
 * Add this guard at the top of any per-user AI processing loop.
 *
 * BEFORE:
 *   for (const userId of userIds) {
 *     await runMemoryConsolidation(userId);

```

```

    * }
    *
    * AFTER:
    */

function isYouTubeUser(userId: string): boolean {
    // TikTok phantom users have IDs starting with 'tiktok_'
    // Real YouTube users have UUID format: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
    if (userId.startsWith('tiktok_')) return false;
    if (userId.startsWith('rumble_')) return false;
    if (userId.startsWith('kick_')) return false;
    if (userId.startsWith('twitch_')) return false;
    return true;
}

// Usage in growth-flywheel, infinite-evolution, self-improvement-engine,
// consistency-agent, knowledge-mesh, vod-seo-optimizer:
//
//   for (const userId of userIds) {
//       if (!isYouTubeUser(userId)) {
//           log.debug(`[GrowthFlywheel] Skipping non-YouTube user: ${userId}`);
//           continue;
//       }
//       await runMemoryConsolidation(userId);
//   }

export { isYouTubeUser };

/**
 * =====
 * FIX #8 – Compliance drift shows count but no detail (20 critical, 3 high)
 * FILE: server/services/automation-engine.ts (or compliance checker)
 * =====
 *
 * The log says "20 critical, 3 high" but no detail on what the violations are.
 * Change the log call to include the actual violation list.
 *
 * BEFORE:
 *   log.warn(`Compliance drift detected: ${critical} critical, ${high} high`);
 *
 * AFTER:
 */

interface ComplianceViolation {
    rule: string;
    severity: 'critical' | 'high' | 'medium' | 'low';
    detail: string;
    videoId?: number | string;
}

```

```
function logComplianceViolations(violations: ComplianceViolation[], log: any) {
  const critical = violations.filter(v => v.severity === 'critical');
  const high     = violations.filter(v => v.severity === 'high');

  if (violations.length === 0) return;

  log.warn(
    `[Compliance] Drift detected: ${critical.length} critical, ${high.length} high`,
    {
      summary: {
        critical: critical.length,
        high:     high.length,
        total:    violations.length,
      },
      criticalViolations: critical.map(v => ({
        rule:    v.rule,
        detail:  v.detail,
        videoId: v.videoId,
      })),
      highViolations: high.map(v => ({
        rule:    v.rule,
        detail:  v.detail,
        videoId: v.videoId,
      })),
    }
  );
}

export { logComplianceViolations };
```

```
/**
 * =====
 * WIRE INTO server/index.ts
 * =====
 *
 * Add to your startup sequence alongside the other services:
 *
 * import { startResurrectionEngine, stopResurrectionEngine }
 *   from './services/resurrection-engine.js';
 *
 * // T+35s – start after other services are initialized
 * setTimeout(() => startResurrectionEngine(), 35_000);
 *
 * // In shutdown handler:
 * stopResurrectionEngine();
 *
 * =====
 * SCHEMA ADDITIONS needed in shared/schema.ts
```

```
*
*
* Add these columns to any table the resurrection engine targets:
*
*   retry_after:          timestamp (nullable) – when to next retry
*   resurrection_count:   integer default 0    – how many times resurrected
*
* For tables that don't have these columns yet, run a migration:
*
*   ALTER TABLE youtube_push_backlog
*     ADD COLUMN IF NOT EXISTS retry_after          TIMESTAMPTZ,
*     ADD COLUMN IF NOT EXISTS resurrection_count   INTEGER DEFAULT 0;
*
*   ALTER TABLE pre_encoder_queue
*     ADD COLUMN IF NOT EXISTS retry_after          TIMESTAMPTZ,
*     ADD COLUMN IF NOT EXISTS resurrection_count   INTEGER DEFAULT 0;
*
*   ALTER TABLE shorts_clip_queue
*     ADD COLUMN IF NOT EXISTS retry_after          TIMESTAMPTZ,
*     ADD COLUMN IF NOT EXISTS resurrection_count   INTEGER DEFAULT 0;
*
*   ALTER TABLE content_vault_backups
*     ADD COLUMN IF NOT EXISTS retry_after          TIMESTAMPTZ,
*     ADD COLUMN IF NOT EXISTS resurrection_count   INTEGER DEFAULT 0;
*
*   ALTER TABLE video_metadata_sync_queue
*     ADD COLUMN IF NOT EXISTS retry_after          TIMESTAMPTZ,
*     ADD COLUMN IF NOT EXISTS resurrection_count   INTEGER DEFAULT 0;
*/
```